

E-Commerce Lakehouse *Technical Documentation*

Databricks Free Edition · Delta Lake · PySpark

Medallion Architecture · SCD Type 2+6 · PII Tokenization · Data Quality

GitHub: <https://github.com/alexsrocha82/ecommerce-lakehouse> | Branch: dev

Table of Contents

1.	Project Overview	3
2.	Architecture — Medallion Pattern	4
3.	Environment Setup (00_setup)	5
4.	Bronze Layer — Auto Loader Ingestion	6
5.	Silver Layer — Orders Quality + MERGE (03_silver_orders)	7
6.	Silver Layer — Customers SCD2+6 + PII (04_silver_customers_scd2)	10
7.	Gold Layer — Star Schema (05_gold_all)	14
8.	Quality Monitoring + Analytics (06_quality_and_analytics)	16
9.	Maintenance — OPTIMIZE + VACUUM (07_optimize)	18
10.	Surrogate Keys — Why Not Used	19
11.	Job Scheduling	20
12.	Running the Pipeline	21

1. Project Overview

This project implements a production-grade data lakehouse pipeline for an e-commerce domain, running entirely on Databricks Free Edition with no Azure credits required. It demonstrates the complete Medallion Architecture — from raw file ingestion to analytical gold tables — using industry-standard tools and patterns.

What you will practice

Topic	Concepts covered
Auto Loader	Incremental JSON ingestion with cloudFiles, schema evolution, availableNow trigger
Delta Lake	ACID transactions, MERGE with partition pruning, OPTIMIZE, ZORDER, time travel
SCD Type 2+6	Full customer history with current_segment denormalization for analytics
Data Quality	6-dimension framework, quarantine pattern, quality score, governance log
PII Protection	SHA-256 tokenization with salt for all sensitive fields — reversible with salt
Star Schema	fact_order_line + dim_customer/product/date — no PII, no surrogate keys
SQL Analytics	Window functions: MoM growth, YTD, DENSE_RANK, churn risk segmentation

2. Architecture — Medallion Pattern

The pipeline follows the Medallion Architecture: three concentric layers of increasing data quality and semantic richness. Each layer has a clear contract — what it stores, why, and who can access it.



Figure 1 — Medallion architecture: Bronze (raw) → Silver (clean) → Gold (analytical)

Bronze Layer — dev_bronze.*

Raw data exactly as it arrived. Append-only. Never updated or deleted. Traceability metadata added (`_source_file`, `_ingested_at`, `_execution_date`). Partitioned by `ingest_year/month`. This is your audit trail.

Silver Layer — dev_silver.*

Typed, deduplicated, quality-validated data. MERGE pattern (no full rewrites). PII replaced with SHA-256 tokens. SCD Type 2+6 for customer history. Failed records isolated in `dev_quarantine.*`

Gold Layer — dev_gold.*

Star schema optimized for analytics. No PII — only `customer_token`. No surrogate keys (see section 10). ZORDER applied for query performance. Joined dimensions: `dim_date`, `dim_customer`, `dim_product`.

Quarantine Layer — dev_quarantine.*

Records that failed quality rules. Preserved with `_dq_errors` array listing which rules failed per row. Team can investigate and reprocess.

Governance Layer — dev_governance.*

Quality logs and monitoring history. Enables SLO tracking, regression detection, and trend analysis across pipeline runs.

3. Environment Setup (00_setup)

Run this notebook exactly once before executing any other notebook. It provisions the entire environment: Unity Catalog Volume, Medallion schemas, folder structure, synthetic data, and the products reference table.

Why Unity Catalog Volumes instead of DBFS?

The Databricks Free Edition disables the DBFS root (error SQLSTATE: 56038 — DBFS_DISABLED). Unity Catalog Volumes are the modern replacement — same API, better governance, supported in all tiers including Free Edition.

```
UC_CATALOG      = 'main'
UC_VOLUME_SCHEMA = 'ecommerce_dev'
UC_VOLUME_NAME   = 'landing'
VOLUME_BASE      = f'/Volumes/{UC_CATALOG}/{UC_VOLUME_SCHEMA}/{UC_VOLUME_NAME}'
```

Folder structure created

```
/Volumes/main/ecommerce_dev/landing/
orders/
  orders_batch_01.json  (200 records)
  orders_batch_02.json  (200 records)
  orders_batch_03.json  (100 records)
customers/
  customers_batch_01.json (100 records)
  customers_batch_02.json (100 records)
checkpoints/
  orders/bronze/      <- Auto Loader state
  customers/bronze/   <- Auto Loader state
```

Why products go directly to Silver

Products are a static reference entity — they arrive already clean and structured from a product catalog system (ERP/PIM). There is no raw version to preserve. Transient data (orders, customers) goes through Bronze because we need the raw audit trail. Reference data that arrives clean goes directly to Silver.

4. Bronze Layer — Auto Loader Ingestion

Notebooks 01 and 02 use Structured Streaming with Auto Loader (cloudFiles format) to ingest JSON files from the landing Volume into Delta Tables.

How Auto Loader works with availableNow=True

Despite using readStream/writeStream, the trigger availableNow=True makes the stream behave as an incremental batch, not a continuous stream:



Figure 2 — Auto Loader incremental batch flow

```
stream = (
    spark.readStream
        .format('cloudFiles')
        .option('cloudFiles.format', 'json')
        .option('cloudFiles.schemaLocation', SCHEMA_PATH)
        .option('cloudFiles.useNotifications', 'false') # directory listing
        .load(LANDING_PATH)
        .withColumn('_source_file', F.col('_metadata.file_path')) # UC Volumes
        .withColumn('_ingested_at', F.current_timestamp())
)
query = stream.writeStream
    .trigger(availableNow=True) # process pending files then stop
    .toTable(BRONZE_TABLE)
```

Key behaviors

- Checkpoint tracks which files were already processed — re-running the notebook never duplicates rows
- Schema inference + evolution: new columns in incoming files are automatically added
- `_metadata.file_path` used instead of `input_file_name()` (required for Unity Catalog)
- Partitioned by `ingest_year/month` for efficient time-range queries
- Bronze is append-only — no updates, no deletes, ever

5. Silver Layer — Orders Quality + MERGE

Notebook 03 applies the quality framework to bronze orders, isolates bad records into quarantine, and merges valid records into the silver table using partition pruning.

5.1 Quality Framework

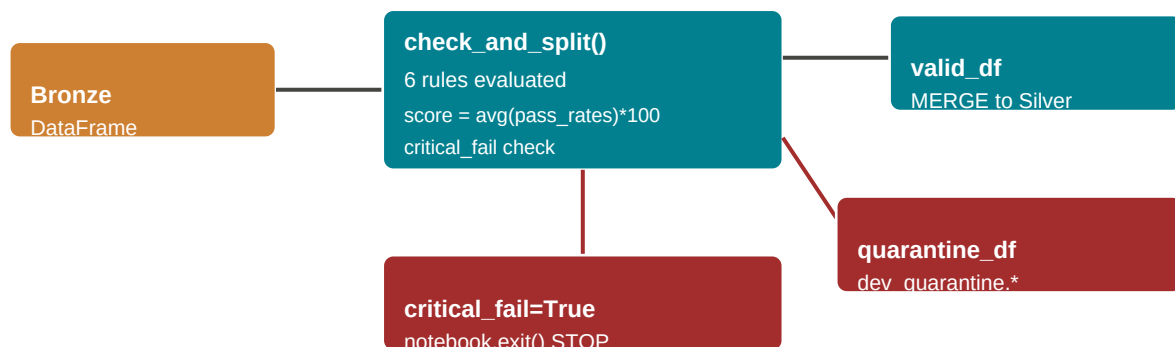
The quality framework is built around two components: the `QualityRule` dataclass and the `check_and_split()` function.

@dataclass — QualityRule

A Python dataclass auto-generates `__init__`, `__repr__`, and `__eq__` from field declarations. It is used here to define each quality rule as a typed object:

```
@dataclass
class QualityRule:
    name: str          # e.g. 'order_id_not_null'
    dimension: str      # completeness | validity | ...
    expression: str     # SQL expression evaluated by F.expr()
    is_critical: bool = False # True = pipeline halts on failure
```

check_and_split() — how it works



critical=True (PK null): pipeline halts, nothing written to Silver

critical=False (invalid amount): row goes to quarantine, pipeline continues

Figure 3 — Quality check flow: valid rows to Silver, invalid rows to Quarantine, critical failures halt the pipeline

Score calculation

The quality score is the average of all rule pass rates, expressed as a percentage:

```
# example with 6 rules, one rule at 90%:
pass_rates = [1.0, 1.0, 0.90, 1.0, 1.0, 1.0]
score = (sum(pass_rates) / len(pass_rates)) * 100
# score = 98.3%
# threshold: score < 95 -> status = 'FAIL' (logged, pipeline continues)
# critical_fail = True -> notebook.exit() (pipeline stops immediately)
```

The 6 quality rules applied to bronze orders

Rule	Dimension	Expression	Critical
order_id_not_null	completeness	order_id IS NOT NULL	YES
customer_id_not_null	completeness	customer_id IS NOT NULL	YES
amount_positive	validity	total_amount > 0	no
quantity_positive	validity	quantity > 0	no
valid_status	validity	status IN (5 values)	no
date_not_future	validity	order_date <= today	no

Three scenarios

Scenario 1 — All rules pass (score >= 95%, no critical fail)

valid_df → MERGE to Silver | quarantine_df (if any) → quarantine table | pipeline continues

Scenario 2 — Score < 95%, no critical fail

Invalid rows go to quarantine | valid rows still reach Silver | status=FAIL logged | pipeline continues

Scenario 3 — Critical rule fails (order_id IS NULL)

notebook.exit() executes immediately | MERGE never runs | Silver is untouched | manual investigation required

5.2 Incremental MERGE with Partition Pruning

The MERGE operation is the core write pattern of the Silver layer. It is idempotent — running the same data multiple times produces the same result.

```

target.alias('t')
.merge(
    source.alias('s'),
    # include partition columns to enable partition pruning
    't.order_id = s.order_id '
    'AND t.order_year = s.order_year '
    'AND t.order_month = s.order_month'
)
.whenMatchedUpdate(
    condition='t.status <> s.status OR t.total_amount <> s.total_amount',
    set={'t.status': 's.status', 't.total_amount': 's.total_amount'}
)
.whenNotMatchedInsertAll()
.execute()

```

Why include order_year and order_month in the join?

order_id alone is sufficient for correctness, but including the partition columns in the join predicate tells Delta Lake which partitions to scan. Without them, Delta reads every partition looking for matching order_ids — a full table scan. With them, Delta reads only the partitions present in the incoming data — typically 1-2 partitions instead of 24+ for a 2-year table.

6. Silver Layer — Customers SCD2+6 + PII

Notebook 04 is the most complex in the pipeline. It combines PII tokenization, SCD Type 2 historical tracking, and SCD Type 6 current-state denormalization in a single three-step process.

6.1 PII Tokenization — SHA-256 + Salt

All sensitive customer fields are replaced with deterministic SHA-256 tokens. This is different from masking (which destroys data irreversibly) — tokenization is reversible by anyone who holds the salt.

Why tokenize instead of mask?

Approach	What it produces	Reversible?	Use case
Masking	u***@gmail.com	No	Data never needed again
Tokenization	a3f8c2d1e9b47051	Yes, with salt	Audit, compliance, fraud investigation
Encryption	AES256(email, key)	Yes, with key	Frequent recovery needed

How the same salt tokenizes all PII fields

The salt is a fixed secret prefix. The uniqueness of each token comes from the combination of salt + value — not from the salt alone:

```
SALT = 'ecommerce-study-salt-2024-changeme' # In prod: Azure Key Vault

@F.udf(StringType())
def pseudonymize(value):
    return SHA256(f'{SALT}:{value}').hexdigest()[:16]

# Same function, different outputs because the VALUE differs:
SHA256('salt:CUST-0001')      -> 'a3f8c2d1e9b47051' # customer_token
SHA256('salt:user1@gmail.com') -> 'f7c9a2b4d1e83056' # email_token
SHA256('salt:123.456.789-99') -> '2d8f1c7e4b93a056' # cpf_token
```

How to reverse a token (de-tokenize)

To find the original value behind a token, iterate over all known candidate values and compare their tokens. This is feasible for customer_id (bounded space) but impractical for email/CPF (unbounded space without a candidate list):

```
import hashlib

SALT = 'ecommerce-study-salt-2024-changeme'
TARGET_TOKEN = 'a3f8c2d1e9b47051' # token to reverse

# Reverse customer_id (bounded: CUST-0001 to CUST-9999)
for i in range(1, 10000):
    cid = f'CUST-{i:04d}'
    candidate = hashlib.sha256(f'{SALT}:{cid}'.encode()).hexdigest()[:16]
    if candidate == TARGET_TOKEN:
        print(f'Found: {cid}') # -> CUST-0001
        break

# Reverse email (requires a list of known emails to compare against)
known_emails = ['user1@gmail.com', 'user2@hotmail.com', ...]
for email in known_emails:
    if hashlib.sha256(f'{SALT}:{email}'.encode()).hexdigest()[:16] == TARGET_TOKEN:
        print(f'Found: {email}')
```

Why tokenize customer_id if it is not personally identifiable?

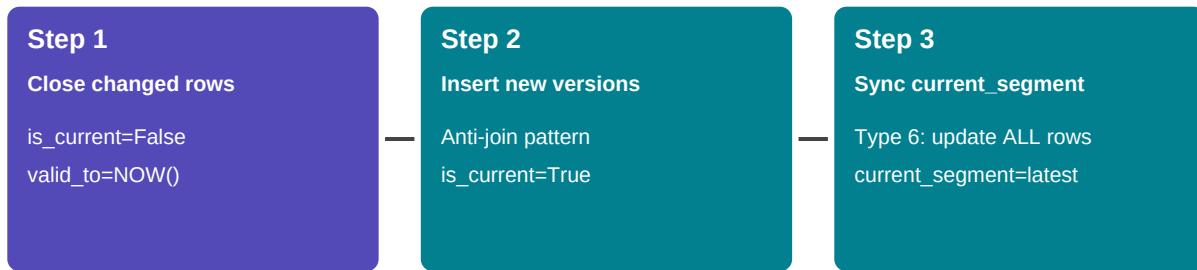
customer_id by itself is not sensitive — CUST-0001 reveals nothing about a person. However, exposing it in the gold layer allows analysts to join the fact table directly with external CRM systems that use customer_id as their primary key. That join would reconstruct full PII (name, email, address) outside the pipeline's control, bypassing all tokenization.

The customer_token is a firewall against joinability — it prevents the gold layer from being used as a bridge to re-identify customers, even accidentally.

Company maturity	Approach	Reason
Less mature (most companies)	customer_id in Gold — RBAC controls access	Analysts don't have CRM access anyway — low practical risk
LGPD/GDPR strict compliance	customer_token required in Gold	Prevents accidental re-identification — audit requirement
Highly sensitive (fintech, healthcare)	customer_token + row-level security + audit log	Every access to PII must be logged and authorized

6.2 SCD Type 2+6 — Three-Step Process

Slowly Changing Dimensions (SCD) is the pattern for tracking how entity attributes change over time. This pipeline implements Type 2 (new row per change) combined with Type 6 (current_segment overwrite on all rows).



segment = value during that period | current_segment = value TODAY

Figure 4 — SCD Type 2+6: three sequential steps per pipeline run

Step 1 — Close changed records (MERGE)

Compares incoming records against active silver rows. If any tracked attribute (email_token, segment, country, customer_name) changed, closes the current row:

```
.whenMatchedUpdate(set={
  't.is_current': 'false',
  't.valid_to': 'current_timestamp()',
})
```

Step 2 — Insert new versions (anti-join)

The anti-join pattern inserts rows only when no active version exists in silver. This covers two cases in one pass: brand-new customers, and customers whose record was just closed in Step 1:

```
current_active = silver.toDF().filter('is_current = true').select('customer_id')

new_versions = incoming
  .join(current_active, 'customer_id', 'left_anti') # only no-active-version rows
  .withColumn('is_current', F.lit(True))
  .withColumn('valid_from', F.current_timestamp())
  .withColumn('valid_to', F.lit(None).cast('timestamp'))
```

Step 3 — Sync current_segment (Type 6 / Type 1 overwrite)

Updates current_segment on ALL rows for each customer — historical and current alike. This is the Type 1 component inside Type 6: a controlled overwrite of one column that enables current-state reporting without joining to a separate lookup:

```
silver.alias('t')
.merge(incoming.select('customer_id', 'segment').alias('s'),
      't.customer_id = s.customer_id')
.whenMatchedUpdate(set={'t.current_segment': 's.segment'})
.execute()
```

What segment vs current_segment answers

```
-- Point-in-time: what segment was this customer in January 2024?
select segment from customers_scd2
where customer_id = 'CUST-0001'
   and valid_from <= '2024-01-15'
   and (valid_to > '2024-01-15' or valid_to is null)

-- Current state: what segment is this customer today?
select current_segment from customers_scd2
where customer_id = 'CUST-0001'
   and is_current = true
```

7. Gold Layer — Star Schema (05_gold_all)

Notebook 05 builds the complete gold layer: dim_date (programmatic), dim_customer (from SCD2 silver), dim_product (from reference silver), and fact_order_line (joined from silver orders + all dimensions).

Table structure

Table	Rows	Key columns	Notes
fact_order_line	500	customer_token, product_id, order_date	No PII, no customer_id
dim_customer	200	customer_token	current records only
dim_product	8	product_id	static reference
dim_date	4018	full_date, date_sk	2020-2030, BR holidays

fact_order_line — the join strategy

Because customer_id was excluded from dim_customer (no PII in gold), the join uses a bridge lookup to resolve customer_id to customer_token before joining:

```
# bridge: silver SCD2 has both customer_id and customer_token
bridge = silver_customers.filter('is_current=true').select('customer_id','customer_token')
silver_orders = silver_orders.join(bridge, 'customer_id', 'left')

# now join dimensions using token – no raw customer_id reaches gold
gold_fact = silver_orders
    .join(dim_customer, 'customer_token', 'left')
    .join(dim_product, 'product_id', 'left')
```

Derived financial metrics

```
cost_amount    = quantity * unit_cost
margin_amount  = total_amount - cost_amount
margin_pct     = round(margin_amount / total_amount * 100, 2)

# unit_cost is stored in dim_product
# calculated as: (price_min + price_max) / 2 * 0.55
# the 0.55 simulates cost = 55% of avg selling price (45% gross margin)
```

Full vs incremental mode

Mode	Reads	Writes	ZORDER	Use when
full	All silver orders	overwrite (dynamic partition)	Yes	First load, schema changes, full reprocess
incremental	Latest order_date only	append	No	Daily pipeline run

8. Quality Monitoring + Analytics (06_quality_and_analytics)

8.1 Quality Monitoring — Part A

Five tables are monitored on every pipeline run. Each check produces a boolean result stored in `checks_json`. The overall status (HEALTHY / DEGRADED / ERROR) is persisted to `dev_governance.quality_monitoring_log`.

Check	What it detects	Tables
volume_ok	rows >= min_rows (silent pipeline failure)	all
freshness_ok	hours since last update <= SLA threshold	all
pk_unique_ok	distinct(pk) == total rows (duplicates)	all
null_{col}_ok	NULL values in critical columns	all
valid_status_ok	status values outside allowed domain	silver.orders only

SLA thresholds per table

Table	SLA (hours)	Reasoning
bronze.orders	1h	Source data must arrive within 1 hour of generation
bronze.customers	2h	Customer updates are less frequent
silver.orders	2h	1h ingestion + 1h processing budget
silver.customers_scd2	4h	Customer changes are slow-moving
gold.fact_order_line	2h	Dashboards must reflect data within 2 hours

Trend detection with Window LAG

The monitoring log stores one row per check per run. A Window LAG detects when a table's status changes between runs — enabling regression alerts:

```
w = Window.partitionBy('table').orderBy('checked_at')
regressions = history
    .withColumn('prev_status', F.lag('status').over(w))
    .filter(col('status') != col('prev_status'))
# HEALTHY -> DEGRADED = regression alert
# DEGRADED -> HEALTHY = recovery notification
```

8.2 Analytics Queries — Part B

Four analytical queries simulate what Power BI or Synapse Serverless would consume from the gold layer. All use PySpark window functions.

B1 — Monthly revenue with MoM growth and YTD

LAG window partitioned by segment ordered by year+month computes previous month revenue. rowsBetween(unboundedPreceding, currentRow) accumulates YTD within the year partition.

B2 — Top 10 customers by revenue (DENSE_RANK)

DENSE_RANK vs RANK: when two customers tie, RANK skips the next position (1,1,3). DENSE_RANK does not skip (1,1,2) — fairer for revenue ranking. PERCENT_RANK assigns VIP tiers: top 1% = VVVIP, top 5% = VVIP, top 20% = VIP.

B3 — Revenue by category with % of total

rowsBetween(unboundedPreceding, unboundedFollowing) without partitionBy creates a window spanning the entire dataset — every row gets the grand total for division.

B4 — Churn risk segmentation (relative threshold)

LAG per customer computes days between consecutive orders. Average cadence is then compared to days since last order. High risk = 3x own cadence, Medium = 2x, Watch = 30 days. This is more accurate than a fixed 90-day rule because it respects each customer's behavior.

9. Maintenance — OPTIMIZE + VACUUM (07_optimize)

Each incremental append creates a small Parquet file. Over time these accumulate and degrade query performance. OPTIMIZE compacts them and ZORDER physically reorders data for faster filtering.

Why OPTIMIZE is NOT run on every pipeline execution

OPTIMIZE rewrites all files in the table — not just the new ones. Running it daily on a small append (500 rows) in a table of 10 million rows means paying the cost of rewriting 10M rows to optimize 500. The right pattern is:

```
daily pipeline:  append new data  (fast, cheap, no OPTIMIZE)
weekly job:      OPTIMIZE + ZORDER (compact accumulated small files)
```

Auto Optimize vs manual OPTIMIZE

Databricks has an Auto Optimize feature that triggers compaction automatically after each write. In paid tiers this works well. In Free Edition it is intentionally disabled in this project because the workspace has a daily compute quota — Auto Optimize would fire on every incremental append and consume quota on low-value compactations. Manual weekly OPTIMIZE gives full control.

Note: In production (paid tier), enable per table: ALTER TABLE ... SET TBLPROPERTIES (delta.autoOptimize.optimizeWrite=true, delta.autoOptimize.autoCompact=true)

ZORDER column selection

ZORDER columns should match the most common filter and join patterns. The most selective column goes first:

Table	ZORDER columns	Reason
fact_order_line	customer_token, order_date, category	Most common analytics filters
dim_customer	customer_token	Join key from fact
dim_product	category, brand	Category/brand filters in B3
silver.orders	order_date, status	MERGE partition pruning + analytics
silver.customers	customer_token, is_current	is_current=true filter in SCD

VACUUM — removing old file versions

Delta Lake retains old Parquet files for time travel. VACUUM removes files older than the retention window. The minimum safe value is 168 hours (7 days) — reducing below this breaks time travel and is blocked by Delta by default.

Note: Set retention to match your recovery SLA. Need 30-day point-in-time recovery? Set vacuum_hours=720. Storage cost concern? Keep 168h and rely on backups.

10. Surrogate Keys — Why Not Used

The gold dimensions do not use integer surrogate keys (SK). This is an intentional architectural decision based on the differences between legacy data warehouses and modern lakehouse platforms.

What surrogate keys are and why they were invented

In the Kimball dimensional modeling methodology (1990s-2000s), fact tables join to dimension tables via integer surrogate keys instead of natural business keys. The motivation was performance: legacy columnar databases like Teradata, Netezza, and early Redshift had significant performance gaps between integer joins and string joins. An integer SK join was often 10x faster than a string join on an email address or customer ID.

SKs also provided isolation: if the source system changed a `customer_id`, the SK in the fact table remained stable — no cascade update needed.

Why SKs are unnecessary in modern Delta Lake

- Delta Lake with ZORDER optimizes data locality for string join columns — performance difference between string and integer joins is negligible
- Databricks has predicate pushdown and bloom filters that make column-level filtering on strings highly efficient
- `monotonically_increasing_id()` in PySpark does not generate sequential or consistent values across executions — it is unreliable as a stable SK
- The natural key `customer_token` is already stable and anonymous — it serves the isolation purpose without requiring a separate SK
- SCD isolation (protecting the fact from source key changes) is handled by the SCD2 pattern in the silver layer, not by a surrogate key in gold

Note: This decision is documented here for reference. In projects targeting legacy BI tools (SSAS, older Tableau versions) that rely on integer SK joins for performance, surrogate keys may still be appropriate.

11. Job Scheduling

The pipeline is split into two Databricks Jobs with independent schedules. Failure in Job 2 never blocks Job 1.

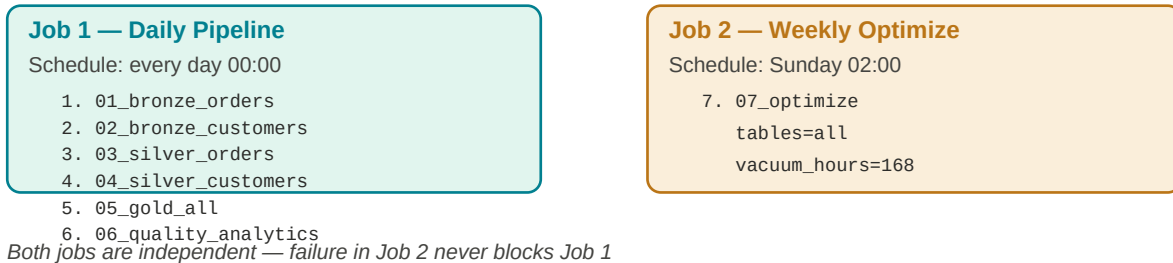


Figure 5 — Two independent Databricks Jobs

Job 1 — Daily pipeline configuration

```
Name      : ecommerce-lakehouse-daily
Schedule  : 0 0 * * * (every day 00:00)
Timezone  : America/Sao_Paulo
Cluster   : Serverless

Tasks (in order, each depends on previous):
Task 1: 01_bronze_orders    param: env=dev, execution_date={{job.start_time.iso_date}}
Task 2: 02_bronze_customers param: env=dev, execution_date={{job.start_time.iso_date}}
Task 3: 03_silver_orders    param: env=dev, execution_date={{job.start_time.iso_date}}
Task 4: 04_silver_customers param: env=dev, execution_date={{job.start_time.iso_date}}
Task 5: 05_gold_all         param: env=dev, mode=incremental
Task 6: 06_quality_analytics param: env=dev
```

Job 2 — Weekly optimize configuration

```
Name      : ecommerce-lakehouse-optimize
Schedule  : 0 2 * * 0 (Sunday 02:00)
Timezone  : America/Sao_Paulo
Cluster   : Single node (I/O-bound, not compute-bound)
Timeout   : 60 minutes

Tasks:
Task 1: 07_optimize param: env=dev, tables=all, dry_run=false, vacuum_hours=168
```

Note: `{{job.start_time.iso_date}}` is a Databricks dynamic variable that resolves to the job execution date in yyyy-MM-dd format — no hardcoding needed.

12. Running the Pipeline

First run — full load

1. Upload all notebooks to Databricks Workspace
2. Run 00_setup (Run All) – ENV=dev
3. Run 01_bronze_orders – ENV=dev
4. Run 02_bronze_customers – ENV=dev
5. Run 03_silver_orders – ENV=dev
6. Run 04_silver_customers_scd2 – ENV=dev
7. Run 05_gold_all – ENV=dev, mode=full
8. Run 06_quality_and_analytics – ENV=dev
9. Run 07_optimize – dry_run=true, then dry_run=false

Expected final state

Table	Expected rows	Notes
dev_bronze.orders	500	3 batches from landing
dev_bronze.customers	200	2 batches from landing
dev_silver.orders	500	MERGE, 0 quarantine
dev_silver.customers_scd2	200	first load, 0 historical
dev_silver.products	8	static reference
dev_gold.fact_order_line	500	no PII, no SK
dev_gold.dim_customer	200	current records only
dev_gold.dim_date	4018	2020-2030
dev_gold.dim_product	8	

Differences vs production environment

Component	Production	This project (Free Edition)
Storage	ADLS Gen2	UC Volumes (/Volumes/main/...)
Auto Loader mode	file notification (Event Grid)	directory listing
Orchestration	ADF or Databricks Jobs	manual + Databricks Jobs
Secrets	Azure Key Vault	local session variable
SQL layer	Synapse Serverless / Power BI	Databricks SQL Warehouse
Unity Catalog	full permissions model	Hive Metastore or basic UC

The PySpark code, MERGE logic, SCD patterns, quality framework, and Delta Lake concepts are identical to production. Free Edition is sufficient to learn and demonstrate all architectural patterns.